

CS 240 - PyTorch Tutorial

TAs: Yash

Spring 2025

Instructions:

- This tutorial is part of your in-lab. The first half of the lab is ungraded, and you would be allowed to access your phones, discuss among yourselves, and ask doubts to TAs. In the second half, we will update this PS with the graded portion .
- INSPECTION implies you have to run and tinker with the code for your understanding. TODO-ungraded implies, you have to fill the unimplemented in the code template, meant for you supervision and confidence. Please DON'T just see the code in this document, have a good look at the python files.
- Create a folder on your Desktop with the following directory structure. The folder must contain only the two specified files:
`submission_<roll_number>/`
`|--classifiers.py`
- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

PyTorch $\gg$ NumPy

You have spent weeks using NumPy. It is a numerical computation library, a very good one, but it only runs on the CPU and knows nothing about gradients. PyTorch is the real game here. It supports the same tensor arithmetic you already know, adds transparent GPU acceleration, and most importantly for what follows can automatically differentiate through any computation you write.

This section focuses entirely on the tensor part. Gradients come in Section 2. For now, the goal is simple: learn to read NumPy and write the equivalent PyTorch, fluently and without thinking.

Note that all code files are confined to section1/

What is a tensor? A `torch.Tensor` is not fundamentally different from a NumPy `ndarray`. Both store a contiguous block of typed numbers described by a shape, a stride, and a dtype. Two new attributes in `torch.Tensor` distinguish them:

- `tensor.device` : records whether the data lives in CPU RAM or in GPU VRAM. A NumPy array always lives on the CPU.
- `tensor.grad` : a tensor of the same shape that PyTorch fills in with gradient information during a backward pass. A NumPy array has no such field.

In this section we ignore both attributes entirely. They reappear in Section 2 (`.grad`) and Section 4 (`.device`). Irrespective, we would be using `device` as "cpu" throughout.

1.1 An Easy One [INSPECTION]

Task. Open `easy.py` and read the NumPy and PyTorch implementations side by side. Run the file and confirm all unit tests pass. There is nothing to implement.

Mathematical specification. Given $X \in \mathbb{R}^{B \times N \times D}$, compute ℓ_2 -normalised features

$$\tilde{x}_{b,i} = \frac{x_{b,i}}{\|x_{b,i}\|_2 + \varepsilon},$$

then the batched pairwise cosine-similarity matrix $S \in \mathbb{R}^{B \times N \times N}$,

$$S[b] = \tilde{X}[b] \tilde{X}[b]^\top,$$

and finally the per-batch mean $m_b = \frac{1}{N^2} \sum_{i,j} S_{b,i,j}$, giving $\mathbf{m} \in \mathbb{R}^B$.

Key observation. Beyond the four substitutions above, every NumPy call maps one-to-one to its PyTorch counterpart. **The mathematical structure is identical; only the API surface differs.**

1.2 Vectorisation Workshop [TODO — ungraded]

Goal. Fill in the **three** functions marked **TODO** in `vectorisation_workshop.py`. The remaining three are given as worked examples. Run `python vectorisation_workshop.py` after each TODO, each has a unit test comparing your output to a NumPy reference.

Notation. B is the batch size, N the feature dimension. Indices are zero-based: $i, j \in \{0, \dots, N-1\}$. Shapes follow PyTorch conventions: $x_1, \tilde{x}_1, x_2, \tilde{x}_2 \in \mathbb{R}^{B \times N}$, $z \in \mathbb{R}^{B \times N \times N}$, $x_{\text{flat}} \in \mathbb{R}^{B \times N^2}$.

Pipeline.

1. Data generation + softmax. [TODO]

Sample $x_1 \sim \mathcal{N}(0, 10 \cdot I)$ and $x_2 \sim \text{Uniform}(-100, 100)$, both in $\mathbb{R}^{B \times N}$. Then apply softmax across the feature dimension to each:

$$\tilde{x}_k[b, i] = \frac{\exp(x_k[b, i])}{\sum_j \exp(x_k[b, j])}, \quad k = 1, 2.$$

Return $(\tilde{x}_1, \tilde{x}_2)$.

Hints: `torch.randn`, `torch.rand`, `torch.nn.functional.softmax(x, dim=1)`.

2. Pairwise matrix, noise, and transpose. [TODO]

(a) For each batch index b , $z[b]_{i,j} = \tilde{x}_1[b, i] \cdot \tilde{x}_2[b, j]$, giving $z \in \mathbb{R}^{B \times N \times N}$.

(b) Add i.i.d. $\mathcal{N}(0, 1)$ noise of the same shape.

(c) Swap the last two axes: $z[b] \leftarrow z[b]^\top$. Return z .

Hints: `x1.unsqueeze(2) * x2.unsqueeze(1)` broadcasts to (B, N, N) ; `torch.randn_like(z.transpose(1, 2))`.

3. Flatten. [given]

Flatten each $N \times N$ slice into a vector: $x_{\text{flat}} \in \mathbb{R}^{B \times N^2}$.

Note: call `.contiguous()` before `.view(B, -1)` because of the transpose in Step 2.

4. Sample mean. [given]

$$\hat{\boldsymbol{\mu}} = \frac{1}{B} \sum_{b=1}^B x_{\text{flat}}[b] \in \mathbb{R}^{N^2}.$$

5. Sample covariance. [TODO]

$$\hat{\Sigma} = \frac{1}{B} \sum_{b=1}^B (x_{\text{flat}}[b] - \hat{\boldsymbol{\mu}})(x_{\text{flat}}[b] - \hat{\boldsymbol{\mu}})^\top \in \mathbb{R}^{N^2 \times N^2}.$$

Hints: Subtract $\hat{\boldsymbol{\mu}}$ (shape $(1, N^2)$ after `unsqueeze(0)`) from x_{flat} to get centred matrix c of shape (B, N^2) ; then $\hat{\Sigma} = c.T @ c / B$.

6. Log-likelihood. [given]

Evaluates $\sum_b \log p(x_{\text{flat}}[b] \mid \hat{\boldsymbol{\mu}}, \hat{\Sigma})$ under the fitted Gaussian. A ridge εI ($\varepsilon = 10^{-6}$) stabilises inversion. *Key functions:* `torch.linalg.inv`, `torch.linalg.slogdet`.

Key observation. No loops meant to exist anywhere. Steps 1–3 transform raw samples into a structured matrix representation; Steps 4–6 fit a Gaussian to that representation. **Every operation is a one- or two-liner in PyTorch once you identify the right axis to act on.**

No One Likes Derivatives

You have just spent the previous section translating NumPy code into PyTorch almost mechanically. The forward computations are the interesting part, they express the mathematical idea you care about. The backward computations are the *mechanical* part, they are determined entirely by the forward code via the chain rule.

This section makes that contrast visceral. We start with a function where the hand-coded derivative is genuinely unpleasant, then show how PyTorch eliminates it in two lines. We then repeat the exercise on a real loss function from logistic regression, and finish with a complete training loop written both ways.

All code lives in `section2/`.

2.1 Warm-up: A Scalar Function

[INSPECTION]

The function. Consider

$$f(x) = \frac{\sin(x^2) e^{-x}}{1 + x^2}.$$

Implementing the *forward* pass takes three lines in either NumPy or PyTorch — they are practically identical:

```
1 # NumPy
2 def f_numpy(x):
3     return np.sin(x**2) * np.exp(-x) / (1 + x**2)
4
5 # PyTorch (spot the difference: torch.* instead of np.*)
6 def f_torch(x):
7     return torch.sin(x**2) * torch.exp(-x) / (1 + x**2)
```

Listing 1: Forward pass — NumPy vs. PyTorch

The hand-coded derivative. Applying the quotient rule to $f = u/v$ with $u = \sin(x^2)e^{-x}$ and $v = 1 + x^2$:

$$f'(x) = \frac{u'v - uv'}{v^2}$$

where

$$u' = e^{-x}(2x \cos(x^2) - \sin(x^2)), \quad v' = 2x.$$

Expanding:

$$f'(x) = \frac{e^{-x}[(2x \cos(x^2) - \sin(x^2))(1 + x^2) - 2x \sin(x^2)]}{(1 + x^2)^2}.$$

Translating this into NumPy correctly takes roughly 5 error-prone lines. Now look at the PyTorch alternative:

```
1 # NumPy (hand-coded -- you must derive and transcribe the formula)
2 def df_numpy(x):
3     u = np.sin(x**2) * np.exp(-x)
4     du = np.exp(-x) * (2*x * np.cos(x**2) - np.sin(x**2))
5     v = 1 + x**2
6     dv = 2*x
7     return (du * v - u * dv) / v**2
8
```

```

9 # PyTorch (autograd -- you write nothing new)
10 x = torch.tensor(x_val, requires_grad=True)
11 y = f_torch(x)      # forward code is identical to f_numpy
12 y.backward()         # PyTorch walks the graph in reverse, applies chain rule
13 grad = x.grad        # done

```

Listing 2: Backward pass — 8 lines of calculus vs. 2 lines of PyTorch

Task. Open `autograd_workshop.py`. Read all four functions in Section 2.1 (`f_numpy`, `df_numpy`, `f_torch`, `df_torch.scalar`). Run the file and observe the plot: the two derivative curves should lie exactly on top of each other. The unit test `test_2_1_derivative_match` verifies this numerically.

Key observation. The forward code was *identical* in effort. The backward code went from 6 dense lines (that are easy to get wrong) to two mechanical lines. *The forward pass is the creative part. The backward pass is mechanical. PyTorch does the mechanical part.*

2.2 Exercise: Loss Function Gradient

[TODO — ungraded]

Setup. Let $\mathbf{X} \in \mathbb{R}^{N \times D}$ be a feature matrix, $\mathbf{y} \in \{0, 1\}^N$ a binary label vector, and $\mathbf{w} \in \mathbb{R}^D$ the weight vector. Define

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \left(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i \right)^2, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Hand-coded gradient (given, read and understand). Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ and the chain rule:

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{2}{N} \mathbf{X}^\top \left[(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y}) \odot \sigma(\mathbf{X}\mathbf{w}) \odot (\mathbf{1} - \sigma(\mathbf{X}\mathbf{w})) \right].$$

This is implemented for you in `grad_loss_numpy`. Note that $\odot$ represent elementwise multiplication.

Your task. Open `autograd_workshop.py` and locate `loss_and_grad_torch`. Fill in the two TODOs:

```

1 def loss_and_grad_torch(w, X, y):
2     # TODO 1: compute L(w) in PyTorch
3     # Translate loss_numpy line-by-line.
4     # Use torch.sigmoid and torch.mean.
5     loss = None # <- replace
6     raise NotImplementedError(...)
7
8     # TODO 2: obtain dL/dw via autograd
9     # (a) zero any existing gradient: if w.grad is not None: w.grad.
10    zero_()
11    # (b) call loss.backward()
12    # (c) copy the gradient: grad = w.grad.clone()
13    grad = None # <- replace
14    raise NotImplementedError(...)
15
16    return loss, grad

```

Listing 3: Skeleton — fill in the two TODOs

Run `python autograd_workshop.py`. All three should print a checkmark once your TODOs are filled in.

Contrast. Compare the gradient you wrote (TODO 2: two lines) with `grad_loss_numpy` (five lines of dense index algebra). Both express exactly the same computation. Only one required you to think.

2.3 Logistic Regression Without Any Abstractions [INSPECTION]

Setup. Open `logistic_regression_raw.py`. It trains logistic regression on the *two-moons* dataset twice using the same hyper-parameters and the same starting weights to produce two comparable runs.

```

1  for epoch in range(N_EPOCHS):
2      # Forward
3      logits = X_np @ w_A + b_A
4      y_hat = sigmoid(logits)
5      loss = bce(y_hat, y_np)
6
7      # Backward (HAND-CODED -- 4 lines)
8      err = y_hat - y_np
9      dw = (X_np.T @ err) / N      # dL/dw = (1/N) X^T (y_hat - y)
10     db = err.mean()              # dL/db = (1/N) sum(y_hat - y)
11
12     # Update
13     w_A -= lr * dw
14     b_A -= lr * db

```

Listing 4: Version A: NumPy logistic regression

The gradient formulas used above (stated without re-derivation):

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}), \quad \frac{\partial L}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i).$$

```

1  for epoch in range(N_EPOCHS):
2      # Forward (identical maths, different ops)
3      logits = X_t @ w_B + b_B
4      y_hat = torch.sigmoid(logits)
5      loss = -torch.mean(y_t * torch.log(y_hat+eps)
6              + (1-y_t) * torch.log(1-y_hat+eps))
7
8      # Backward (AUTograd -- replaces the 4-line block above)
9      if w_B.grad is not None: w_B.grad.zero_()
10     if b_B.grad is not None: b_B.grad.zero_()
11     loss.backward()
12
13     # Update (same formula, slightly different syntax)
14     with torch.no_grad():
15         w_B -= lr * w_B.grad
16         b_B -= lr * b_B.grad

```

Listing 5: Version B: PyTorch autograd replaces the backward block

Side-by-side summary.

Block	Version A (NumPy)	Version B (PyTorch)
Forward	<code>np.*</code> operations	<code>torch.*</code> — nearly identical
Backward	4 lines of index algebra	<code>loss.backward()</code>
Update	<code>w -= lr * dw</code>	<code>with torch.no_grad(): w -= lr * w.grad</code>

Task. Read both versions, run the file, and verify: (1) the final accuracy of both versions is similar, (2) the decision boundaries in the saved plot look the same, (3) the training curves overlap. The file prints a summary and saves `2_3_logistic_regression_raw.png`.

Teaser. Version B still has raw tensors floating around. We are managing `.grad.zero_()` manually, writing the update rule by hand, and cannot easily swap optimisers or move parameters to a GPU. Every ML pipeline has this same bookkeeping mess.

We have raw tensors floating around. We're managing `.grad.zero_()` by hand. We're writing the update rule explicitly. Every ML pipeline has this same structure. Can we do better? → Section 3 names the abstractions that clean this up.

The Classic ML Pipeline

Section 2 ended with a complete, working training loop raw PyTorch, no abstractions. Before we start adding layers of abstraction, step back and look at the *structure* of what we already wrote. Every supervised learning pipeline you will ever write has exactly the same three moving parts. This section names them, dissects an example to find them, and maps each one to the PyTorch abstraction that cleans it up.

There are no exercises here. Read, absorb the picture, and move on.

3.1 The Annotated Pipeline

A miniature example. Below is a complete, raw training run for a two-layer neural network classifier on the two-moons dataset. No `nn.Module`, no `optim`, no `DataLoader`. Just tensors, arithmetic, and `.backward()`.

Most helper code is condensed with `...` what matters is the *shape* of the computation, not every implementation detail.

```
1 import torch, numpy as np
2 from sklearn.datasets import make_moons
3
4 # helpers
5 def relu(z): return z.clamp(min=0)
6 def sigmoid(z): return 1 / (1 + torch.exp(-z))
7 def bce(yhat, y, eps=1e-9): # this here computes the model loss
8     ...
9     return -torch.mean(y*torch.log(yhat+eps) + (1-y)*torch.log(1-yhat+eps))
10 )
```

```

10
11 #           data
12 X_np, y_np = make_moons(n_samples=500, noise=0.25, random_state=0)
13 X_np = (X_np - X_np.mean(0)) / X_np.std(0)
14 # ... train/val split, shuffle indices, form mini-batches ...
15 X = torch.tensor(X_np, dtype=torch.float32)
16 y = torch.tensor(y_np, dtype=torch.float32)
17
18 #           model
19 H = 16 # hidden width
20 W1 = torch.randn(2, H, requires_grad=True) * 0.1
21 b1 = torch.zeros(H, requires_grad=True)
22 W2 = torch.randn(H, 1, requires_grad=True) * 0.1
23 b2 = torch.zeros(1, requires_grad=True)
24
25 params = [W1, b1, W2, b2]
26
27 #           training loop
28 lr = 0.05
29 for epoch in range(300):
30     # forward
31     h = relu(X @ W1 + b1) # (N, H)
32     yhat = sigmoid(h @ W2 + b2).squeeze() # (N,)
33     loss = bce(yhat, y)
34
35     # backward
36     for p in params:
37         if p.grad is not None: p.grad.zero_()
38     loss.backward()
39
40     # update
41     with torch.no_grad():
42         for p in params:
43             p -= lr * p.grad
44
45     if epoch % 50 == 0:
46         ... # log accuracy

```

Listing 6: A complete raw training pipeline (abbreviated).

Finding the structure. Read the listing again with fresh eyes. Three distinct jobs are happening, and they never overlap:

1. Lines 11–16 exist entirely to produce tensors X and y . Everything else is agnostic to where that data came from.
2. Lines 18–25 *define* the model, what its parameters are and how a forward pass is computed.
3. Lines 27–38 *run* the model against data, measure error, compute gradients, and move the parameters in the right direction.

The three blocks are annotated below.

DATA — what goes in

```
X_np, y_np = make_moons(n_samples=500, noise=0.25, random_state=0)
X_np = (X_np - X_np.mean(0)) / X_np.std(0)    # zero mean, unit variance
# ... shuffle, split 80/20, form mini-batches of size 32 ...
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32)
```

This block answers: *what is the data and how is it shaped before training sees it?* It covers raw loading, normalisation, splitting, and conversion to tensors. In a real project it also contains augmentation, class-balancing, caching, and parallel loading.

As written, this logic is tangled with the training script. Swapping the dataset (say, from two-moons to MNIST) requires editing the training file.

This block becomes `torch.utils.data.Dataset + DataLoader`.

MODEL — what is being trained

```
H = 16
W1 = torch.randn(2, H, requires_grad=True) * 0.1
b1 = torch.zeros(H, requires_grad=True)
W2 = torch.randn(H, 1, requires_grad=True) * 0.1
b2 = torch.zeros(1, requires_grad=True)
params = [W1, b1, W2, b2]                # collected manually

def forward(X):
    h = relu(X @ W1 + b1)
    yhat = sigmoid(h @ W2 + b2).squeeze()
    return yhat
```

This block answers: *what are the learnable quantities, and how does a forward pass map inputs to predictions?*

As written, there is nothing stopping you from accidentally using a parameter without adding it to `params`, forgetting `requires_grad=True` on one tensor, or having two copies of `params` lists that drift out of sync. Moving the model to a GPU requires calling `.cuda()` on every parameter individually. Saving/loading the model has no standard interface.

All in all, we want to treat model parameter as a whole, and define the model logic in one place.

This block becomes `torch.nn.Module`.

TRAINING — the learning engine

```
for epoch in range(300):
    yhat = forward(X)
    loss = bce(yhat, y)

    for p in params:
        # zero old gradients
        if p.grad is not None: p.grad.zero_()
    loss.backward()          # compute new gradients

    with torch.no_grad():
```

```

for p in params:                # SGD update rule (hand-written)
    p -= lr * p.grad

```

This block answers: *how do we adjust the parameters to reduce the loss?* It contains the zero-grad / backward / update cycle that is identical in structure for every gradient-based model ever trained.

As written, the update rule (SGD) is hard-coded. Switching to Adam (another optimisation method like SGD) requires rewriting the update block and tracking momentum buffers by hand. The zero-grad boilerplate is repeated in every project.

This block becomes `torch.autograd + torch.optim`.

3.2 The Three Abstractions

Each abstraction is a clean interface over one of the three blocks above. Here is just enough to know what each one *is* before we dig in.

`torch.nn.Module` : The Model.

You subclass `nn.Module`, define `__init__` (declare parameters and sub-layers) and `forward` (write the forward pass once). In exchange you get: `.parameters()` to iterate over every learnable tensor without maintaining a list yourself; `.to(device)` to move the entire model to a GPU in one call; `print(model)` for a readable architecture summary; `torch.save` / `torch.load` for checkpointing. The key guarantee is that any tensor declared as an `nn.Parameter` (or wrapped in a child module) is automatically registered and tracked.

`torch.autograd + torch.optim` : the training engine.

Autograd replaces hand-coded gradients: the forward pass builds a computation graph implicitly, and `loss.backward()` traverses it in reverse to fill `.grad` on every leaf tensor. Optim replaces hand-coded update rules: `optimizer.step()` iterates over `model.parameters()` and applies the chosen rule (SGD, Adam, RMSProp, ...). Switching from SGD to Adam is a one-line change; the training loop is untouched. `optimizer.zero_grad()` replaces the manual loop over `params`.

`torch.utils.data.Dataset + DataLoader` : the data pipeline.

`Dataset` defines what the data *is*: implement `__len__` (total number of samples) and `__getitem__(i)` (return the *i*-th sample as a tensor pair). `DataLoader` defines how to *iterate*: it draws indices, calls `__getitem__` for each, and collates them into batched tensors with optional shuffling and parallel prefetching. This is the natural home for all messy preprocessing: pixel normalisation ($[0, 255] \rightarrow [0, 1]$), colour-space conversion, data augmentation, train/val splitting. Decoupling it from the training loop means you can swap datasets without touching a single line of training code.

torch.nn.Module: The Model Engine

Section 2 showed that autograd eliminates hand-coded derivatives. Section 3 named the three abstractions that structure every ML pipeline: data, model, and training loop. This section builds the *model* abstraction: `nn.Module`.

A module is a Python object that owns parameters, knows how to compute a forward pass, and integrates cleanly with optimisers and devices. We start with logistic regression on two-moons, which you already know, and build upward from there. All code lives in [section4/](#).

4.0 Logistic Regression as an `nn.Module` [INSPECTION]

Two ways to hold a model. In Section 2 the weights `w` and `b` were plain tensors created by hand. An `nn.Module` wraps that same idea in a class. Compare the two styles on logistic regression:

```
1 class LogisticRegressionNumPy:
2     def __init__(self, input_dim, lr=0.1, seed=0):
3         rng = np.random.default_rng(seed)
4         self.w = rng.normal(0, 0.01, (input_dim,)).astype(np.float32)
5         self.b = np.float32(0.0)
6         self.lr = lr
7
8     def predict_proba(self, X):
9         return sigmoid_np(X @ self.w + self.b)
10
11    def fit(self, X, y, n_epochs=300):
12        for _ in range(n_epochs):
13            y_hat = self.predict_proba(X)
14            loss = bce_np(y_hat, y)
15            # hand-coded gradients every time
16            err = y_hat - y
17            dw = (X.T @ err) / len(y)
18            db = err.mean()
19            self.w -= self.lr * dw
20            self.b -= self.lr * db
```

Listing 7: NumPy class : parameters are plain arrays managed by hand.

```
1 class LogisticRegressionTorch(nn.Module):
2     def __init__(self, input_dim):
3         super().__init__() # this is important
4         # nn.Linear stores weight (1, input_dim) and bias (1,) as
5         # nn.Parameter objects, they appear in model.parameters()
6         # automatically, with requires_grad=True.
7         self.linear = nn.Linear(input_dim, 1)
8
9     def forward(self, x):
10        logit = self.linear(x) # (N, 1)
11        return torch.sigmoid(logit).squeeze(1) # (N,) <-- probabilities
```

Listing 8: PyTorch `nn.Module` : parameters are registered automatically; `forward()` is pure mathematics.

What changed. Three things disappeared:

- *Manual weight storage* : `nn.Linear` holds weight and bias internally as `nn.Parameter` objects.
- *Manual gradient code* : `forward()` contains no derivative logic at all. Gradients are computed by autograd when the caller calls `loss.backward()`.
- *Manual update rule* : the training loop (shown below) calls `optimizer.step()` on behalf of any optimiser.

The training loop on the outside. Because `LogisticRegressionTorch` does not contain any training logic, the caller is responsible for the five canonical lines:

```

1  #          setup

2  model      = LogisticRegressionTorch(input_dim=2)
3  optimizer  = torch.optim.SGD(model.parameters(), lr=0.1)
4  loss_fn    = nn.BCELoss()
5
6  #          training loop

7  for epoch in range(300):
8      y_hat = model(X_t)          # forward pass
9      loss  = loss_fn(y_hat, y_t) # compute loss
10     optimizer.zero_grad()       # clear stale gradients
11     loss.backward()             # compute new gradients
12     optimizer.step()            # update parameters

```

Listing 9: Training loop for `LogisticRegressionTorch` on two-moons.

Checking what the module knows about itself. `nn.Module` provides introspection for free:

```

1  print(model)
2  # LogisticRegressionTorch(
3  #   (linear): Linear(in_features=2, out_features=1, bias=True)
4  # )
5
6  for name, p in model.named_parameters():
7      print(name, p.shape)
8  # linear.weight  torch.Size([1, 2])
9  # linear.bias    torch.Size([1])

```

Listing 10: Introspecting a trained `LogisticRegressionTorch`.

Task. Open `log_reg_module.py`. Read both class definitions, then read the training loop. Run the file and confirm that: (1) both models reach similar accuracy, (2) the training curves overlap in the saved plot, (3) all unit tests print a checkmark.

Key observation. The module contains the *what* (forward pass) but not the *how* (training procedure). This separation is what makes it easy to swap optimisers, loss functions, or hardware without touching the model class.

4.2 Polynomial Logistic Regression

[INSPECTION]

The problem. Two-moons is not linearly separable: no straight line can correctly classify all points. A logistic regression model trained on the raw features achieves roughly 83% accuracy it approximates the boundary with a line across the gap. Adding polynomial features changes the hypothesis class from lines to arbitrary conics.

Feature expansion. Define the monomial expansion of degree d :

$$\phi(\mathbf{x}) = [x_1^a x_2^b \mid a \geq 0, b \geq 0, a + b \leq d], \quad \mathbf{x} = (x_1, x_2) \in \mathbb{R}^2.$$

The number of monomials is $M = \frac{(d+2)(d+1)}{2}$. For $d = 2$: $M = 6$ features $\{1, x_2, x_2^2, x_1, x_1 x_2, x_1^2\}$. Logistic regression on $\phi(\mathbf{x})$ can represent any conic decision boundary.

Architecture. Open `poly_logistic.py` and read `PolynomialLogisticRegression`:

```
1 class PolynomialLogisticRegression(nn.Module):
2     def __init__(self, degree):
3         super().__init__() #this is important
4         self.degree = degree
5         M = (degree + 2) * (degree + 1) // 2
6         self.linear = nn.Linear(M, 1) # ONLY learnable part
7
8     def _expand_features(self, x):
9         # enumerate all (a, b) with a >= 0, b >= 0, a+b <= degree
10        x1, x2 = x[:, 0:1], x[:, 1:2]
11        features = []
12        for a in range(self.degree + 1):
13            for b in range(self.degree + 1 - a):
14                features.append(x1**a * x2**b)
15        return torch.cat(features, dim=1) # (B, M)
16
17    def forward(self, x):
18        phi = self._expand_features(x) # non-learnable
19        return torch.sigmoid(self.linear(phi)).squeeze(1)
```

Listing 11: `PolynomialLogisticRegression`: non-learnable feature map + learnable linear layer.

The key point: `_expand_features` is not an `nn.Module` and not an `nn.Parameter`. It is a deterministic function with no learned weights. `list(model.parameters())` returns only the weight and bias of `self.linear`, exactly $M + 1$ scalars.

Task. Open `poly_logistic.py`. Read `PolynomialLogisticRegression` and the `train` function. Run the file and observe that: (1) `degree=1` gives a linear boundary with lower accuracy, (2) `degree=2` produces a curved boundary that cleanly separates the moons, (3) the saved plot shows both decision boundaries side by side.

Key observation. `forward()` can mix learnable layers and fixed, non-learnable transformations freely. `nn.Module` tracks only the tensors that are registered as `nn.Parameter` or as sub-modules, everything else is transparent to the optimiser.

4.3 Exercise: Feed-Forward Network

[TODO — ungraded]

Setup. A *feed-forward neural network* (also called a multi-layer perceptron) stacks linear layers with non-linear activations between them. The architecture you will implement has two hidden layers and uses the rectified linear unit (ReLU) as the activation function:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{x} \mathbf{W}_1^\top + \mathbf{b}_1), \quad \mathbf{h}_2 = \text{ReLU}(\mathbf{h}_1 \mathbf{W}_2^\top + \mathbf{b}_2), \quad \hat{\mathbf{y}} = \mathbf{h}_2 \mathbf{W}_3^\top + \mathbf{b}_3,$$

where $\mathbf{x} \in \mathbb{R}^{B \times D}$, each hidden layer is $\mathbb{R}^{B \times H}$, and the output is $\mathbb{R}^{B \times C}$. The ReLU is $\text{ReLU}(z) = \max(0, z)$, applied element-wise.

NumPy reference (read first). `FeedForwardNumPy` in `fcnn.py` implements these equations manually. The weights $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$ are NumPy arrays of shapes (H, D) , (H, H) , and (C, H) respectively the same convention as `nn.Linear`, which stores `weight` of shape `(out_features, in_features)`. Read `FeedForwardNumPy.forward()` before writing any PyTorch code.

Your task. Open `fcnn.py` and fill in the two marked blocks in `FeedForwardNN`:

```
1 class FeedForwardNN(nn.Module):
2     def __init__(self, input_dim, hidden_dim, output_dim):
3         super().__init__()
4         # TODO: define three linear layers named fc1, fc2, fc3
5         #   fc1: input_dim -> hidden_dim
6         #   fc2: hidden_dim -> hidden_dim
7         #   fc3: hidden_dim -> output_dim
8         # Use nn.Linear(in_features, out_features).
9         ...
10
11     def forward(self, x):
12         # TODO: pass x through fc1 -> relu -> fc2 -> relu -> fc3
13         # Use F.relu(tensor) for the activation.
14         # Return the output of fc3 directly (raw logits, no sigmoid).
15         ...
```

Listing 12: Skeleton for `FeedForwardNN` : fill in `__init__` and `forward`.

The output of `forward()` must be *raw logits* of shape $(B, \text{output_dim})$ with no sigmoid applied. The caller (or loss function) applies the sigmoid. This is the standard convention when using `BCEWithLogitsLoss`.

Run `python fcnn.py`. All five should print a checkmark once your implementation is complete.

Shorthand: `nn.Sequential`. PyTorch provides a convenience wrapper for architectures where every layer is applied in a fixed sequence with no branching:

```
1 model = nn.Sequential(
2     nn.Linear(input_dim, hidden_dim), nn.ReLU(),
3     nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
4     nn.Linear(hidden_dim, output_dim),
5 )
```

Listing 13: The same two-hidden-layer network written with `nn.Sequential`.

`nn.Sequential` is fine for simple feed-forward stacks. The explicit `nn.Module` subclass becomes necessary as soon as `forward()` needs branching logic, skip connections, shared weights, or non-learnable operations, all of which you will encounter in more advanced architectures.

Our module now holds the parameters and the forward computation. But training still requires an optimiser, a loss function, and a loop that puts the five canonical lines together. → Section 5 equips the module with a training engine.

torch.autograd + torch.optim: The Training Engine

You have already seen how autograd computes gradients. The remaining question is: what do you *do* with them? This section answers that by first showing the manual approach and its limitations, then introducing `torch.optim` as the standard solution.

5.0 The Problem With Manual Updates

[INSPECTION]

Starting point. After Section 4 you have a `LogisticRegression` module. Suppose you write the training loop by hand, updating parameters directly using the gradients autograd has computed:

```
1 model = LogisticRegression(input_dim=2)
2 lr = 0.1
3
4 for epoch in range(200):
5     y_hat = model(X)
6     loss = F.binary_cross_entropy(y_hat, y)
7
8     # zero out accumulated gradients from the previous step
9     for p in model.parameters():
10         if p.grad is not None:
11             p.grad.zero_()
12
13     loss.backward()
14
15     # manually step every parameter
16     with torch.no_grad():
17         for p in model.parameters():
18             p.data -= lr * p.grad
```

Listing 14: Manual SGD on a single batch.

This works for plain SGD with a fixed learning rate. Now suppose you want to do one of the following:

- Decay the learning rate by $0.1\times$ every 50 epochs.
- Switch to *SGD with momentum*, which requires tracking a velocity vector v_t for every parameter across steps.
- Switch to *Adam*, which requires tracking two moment vectors m_t and v_t per parameter, plus a bias-correction schedule.

Each change forces you to (a) maintain extra state tensors alongside every parameter, (b) rewrite the update rule, and (c) get the bookkeeping right yourself. For a model with dozens of parameter

tensors this becomes error-prone and tedious. **The optimiser is a well-defined sub-problem, there is no reason to solve it from scratch in every training script.**

5.1 torch.optim: SGD

[INSPECTION]

What an optimiser object does. A `torch.optim` optimiser holds a reference to the model's parameters and encapsulates the update rule and any associated state. You interact with it through two calls:

- `optimizer.zero_grad()` : clears `.grad` on every tracked parameter (replaces the manual loop above).
- `optimizer.step()` : applies the update rule to every tracked parameter using the current `.grad` values.

```
1 import torch
2 import torch.nn.functional as F
3 from torch import optim
4 from sklearn.datasets import make_moons
5
6 # data
7 X_np, y_np = make_moons(n_samples=500, noise=0.2, random_state=0)
8 X = torch.tensor(X_np, dtype=torch.float32)
9 y = torch.tensor(y_np, dtype=torch.float32)
10
11 # model + optimiser
12 model = LogisticRegression(input_dim=2)
13 optimizer = optim.SGD(model.parameters(), lr=0.1)
14
15 # training loop
16 for epoch in range(200):
17     y_hat = model(X) # forward pass
18     loss = F.binary_cross_entropy(y_hat, y) # compute loss
19     optimizer.zero_grad() # clear old gradients
20     loss.backward() # compute new gradients
21     optimizer.step() # update parameters
22
23     if (epoch + 1) % 50 == 0:
24         print(f Epoch_{epoch+1:3d} loss={loss.item():.4f} )
```

Listing 15: Training LogisticRegression on two-moons with SGD via `torch.optim`.

Key observation. The five-line loop body is identical to what you would write for any model and any optimiser. The call to `optimizer.zero_grad()` replaces the manual loop over `model.parameters()`; `optimizer.step()` replaces the manual parameter update inside `torch.no_grad()`.

5.2 Switching Optimisers

[INSPECTION]

From SGD to Adam. Adam maintains per-parameter first and second moment estimates that adapt the effective learning rate during training. Implementing this by hand requires allocating and updating two extra tensors per parameter at every step. With `torch.optim` the change is a single line:

```
1 # SGD (from 5.1):
2 # optimizer = optim.SGD(model.parameters(), lr=0.1)
```



```

3
4 # Adam      one line changes, the training loop is untouched:
5 optimizer = optim.Adam(model.parameters(), lr=1e-3)
6
7 # The five-line loop body is identical.
8 for epoch in range(200):
9     y_hat = model(X)
10    loss = F.binary_cross_entropy(y_hat, y)
11    optimizer.zero_grad()
12    loss.backward()
13    optimizer.step()

```

Listing 16: Replacing SGD with Adam — only the constructor changes.

Learning rate scheduling. If you want to decay the learning rate, attach a scheduler to the optimiser and call `scheduler.step()` once per epoch:

```

1 optimizer = optim.SGD(model.parameters(), lr=0.1)
2 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=50, gamma=0.1)
3
4 for epoch in range(200):
5     y_hat = model(X)
6     loss = F.binary_cross_entropy(y_hat, y)
7     optimizer.zero_grad()
8     loss.backward()
9     optimizer.step()
10    scheduler.step()           # decays lr at epochs 50, 100, 150

```

Listing 17: StepLR decays the learning rate by a factor every N epochs.

The scheduler mutates `optimizer.param_groups[0]["lr"]` in place. You do not touch the training loop body. **The optimiser and scheduler are clean abstractions: swap algorithms by changing the constructor, not the loop.**

Refer to `training.py` in section5/ for a more coherent version of the above code.

→ Section 6 introduces `DataLoader`, which replaces the full-batch `model(X)` call with a mini-batch loop, the only remaining structural change to the training loop.

torch.utils.data: Datasets and DataLoading Management

[WE SKIP THIS FOR IN-LAB AS WE DO NOT REQUIRE IT. WE WILL UPDATE WHEN WE PUBLISH THE TUTORIAL ON MOODLE.]

Everything In One Frame

This section is individual and graded. TAs are present but will not help with implementation. You may consult your own code from Sections 2–5. Phones and collaboration are not permitted.

You have 60 minutes. You have to only fill the training loop in `classifiers.py`, part of the function `train_classifier(...)`. Run `python classifiers.py` to check your progress locally. When all tests pass, run `check-cs240`, then call a TA to run `submit-cs240`.

7.0 Setting

NASA’s Kepler telescope monitored 150 000 stars for four years, searching for periodic dips in brightness caused by planets transiting in front of their host star. The bottleneck was not data collection but triage: most candidate signals are false positives, background star eclipses, detector artefacts, or noise fluctuations.

Your task is to train a neural network that separates confirmed exoplanet transit candidates from false positives, given six features extracted from each light curve:

Feature	Description	Typical range
<code>transit_depth</code>	Fractional brightness drop	0.002–0.08
<code>transit_duration</code>	Duration of the transit (hours)	1–20
<code>orbital_period</code>	Estimated orbital period (days)	1–500
<code>stellar_radius</code>	Host-star radius (solar radii)	0.1–3
<code>equilibrium_temp</code>	Planet equilibrium temperature (K)	100–2500
<code>signal_to_noise</code>	Detection signal-to-noise ratio	5–100

Label: 0 = false positive, 1 = confirmed candidate.

Three deliberate differences from Sections 4–5.

1. **Six input features with very different scales.** `equilibrium_temp` can exceed 2000 K while `transit_depth` is below 0.1. Gradient descent behaves poorly when features live on different scales. You must standardise before training.
2. **The model returns raw logits, not probabilities.** Do *not* apply `sigmoid` inside `forward`. Use `nn.BCEWithLogitsLoss`, which fuses sigmoid and binary cross-entropy in a single numerically stable operation.
3. **Accuracy requires an explicit sigmoid.** Because the model outputs logits, converting them to class predictions requires applying `torch.sigmoid` first, then thresholding at 0.5.

Mini-batching is handled by the provided `make_batches(X, y, batch_size)` function, use it in place of a `DataLoader`.

7.1 ExoplanetClassifier

[GIVEN - INSPECT]

Implement `ExoplanetClassifier(nn.Module)`.

Building blocks **Linear layer.** `nn.Linear(in_features, out_features)` stores a weight matrix $W \in \mathbb{R}^{out \times in}$ and a bias vector $\mathbf{b} \in \mathbb{R}^{out}$, and computes

$$\text{Linear}(x) = xW^\top + \mathbf{b}, \quad x \in \mathbb{R}^{B \times in}, \quad \text{output} \in \mathbb{R}^{B \times out}.$$

Declare all layers as named attributes inside `__init__` (e.g. `self.fc1 = nn.Linear(...)`) so that PyTorch registers their parameters automatically and they appear in `model.parameters()`.

ReLU activation. Applied element-wise after each *hidden* linear layer:

$$\text{ReLU}(z) = \max(0, z).$$

Use `torch.relu(z)` in `forward`. Do *not* apply ReLU after the final layer, the output must remain an unbounded real number (a logit).

Squeeze. `nn.Linear(16, 1)` produces output of shape $(B, 1)$, a column vector with one entry per sample. Calling `.squeeze(1)` removes that trailing size-1 dimension to give shape $(B,)$. This matters: the loss function and accuracy function both expect a 1-D tensor of length B , not a 2-D tensor of shape $(B, 1)$.

Architecture.

Layer name	Operation	Input shape	Output shape
fc1	<code>nn.Linear(6, 32)</code>	$(B, 6)$	$(B, 32)$
	ReLU	$(B, 32)$	$(B, 32)$
fc2	<code>nn.Linear(32, 16)</code>	$(B, 32)$	$(B, 16)$
	ReLU	$(B, 16)$	$(B, 16)$
fc3	<code>nn.Linear(16, 1)</code>	$(B, 16)$	$(B, 1)$
	<code>.squeeze(1)</code>	$(B, 1)$	$(B,)$

The total trainable parameter count is:

$$\underbrace{6 \cdot 32 + 32}_{\text{fc1}} + \underbrace{32 \cdot 16 + 16}_{\text{fc2}} + \underbrace{16 \cdot 1 + 1}_{\text{fc3}} = 224 + 528 + 17 = 769.$$

Verification. The auto-grader checks: output shape is $(B,)$; output is *not* bounded to $(0, 1)$, a sigmoid anywhere in `forward` will fail this check; parameter count equals 769.

7.2 standardize

[GIVEN - INSPECT]

Implement `standardize(X_train, X_val)`.

Why standardise. The six features span very different numerical ranges. A gradient step that makes progress along the `orbital_period` axis (range ~ 500) shifts parameters by a very different amount than one along `transit_depth` (range ~ 0.08). Standardising puts every feature on the same scale so that gradient descent converges reliably.

The transformation. For each feature column $d \in \{0, \dots, 5\}$, compute the mean $\hat{\mu}_d$ and standard deviation $\hat{\sigma}_d$ across the *training rows only*, then apply:

$$\tilde{x}_{n,d} = \frac{x_{n,d} - \hat{\mu}_d}{\hat{\sigma}_d + 10^{-8}}.$$

The 10^{-8} guards against division by zero if a column has zero variance. After this step, every column of the transformed training matrix has mean ≈ 0 and standard deviation ≈ 1 .

Critical rule, no data leakage. Compute $\hat{\mu}$ and $\hat{\sigma}$ on `X_train` only. Apply the *same* constants to `X_val`. Fitting statistics on the validation set would give an over-optimistic accuracy estimate: the model would indirectly have seen validation data before evaluation.

What to fill in. NO NEED TO FILL. WE HAVE FILLED IT FOR YOU Compute mean and std along `axis=0` of `X_train`. Subtract and divide both arrays using those constants. Return `(X_train_std, X_val_std)` as NumPy arrays of the same shape as the inputs.

Verification. Every column of the returned `X_train_std` has mean ≈ 0 and std ≈ 1 ; `X_val_std` is shifted by the training statistics, not independently re-fitted.

7.3 compute_accuracy

[GIVEN - INSPECT]

Implement `compute_accuracy(logits, labels)`.

From logits to predictions. The model outputs a raw score $z \in \mathbb{R}$ per sample. The sigmoid converts it to a probability:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1).$$

The predicted class is

$$\hat{y} = \mathbf{1}[\hat{p} \geq 0.5] = \mathbf{1}[z \geq 0].$$

Accuracy is the fraction of samples for which $\hat{y} = y$.

What to fill in. Apply `torch.sigmoid` to `logits` to obtain probabilities. Threshold at 0.5 to obtain a Boolean prediction tensor. Compare element-wise to `labels` and return the mean as a Python `float`. A clean idiom:

```
probs          = torch.sigmoid(logits)
predictions    = (probs >= 0.5)
accuracy       = (predictions == labels.bool()).float().mean().item()
```

Verification. Returns 1.0 when all logits are large positive and all labels are 1 (or large negative and all labels 0); returns 0.5 on the half-correct test case.

7.4 train_classifier

[TODO — graded]

Implement `train_classifier(model, X_train, y_train, X_val, y_val, lr, batch_size, num_epochs)`.

What to fill in. Follow these steps in order:

1. **Standardise.** Call `standardize(X_train, X_val)` to obtain normalised NumPy arrays `X_tr`, `X_v`.

2. **Convert to tensors.** Cast all four arrays to `torch.float32` tensors:

```
X_tr = torch.tensor(X_tr, dtype=torch.float32)
y_tr = torch.tensor(y_train, dtype=torch.float32)
X_v  = torch.tensor(X_v,  dtype=torch.float32)
y_v  = torch.tensor(y_val, dtype=torch.float32)
```

3. **Construct loss function and optimiser.** `BCEWithLogitsLoss` expects raw logits. Passing post-sigmoid values to it would be incorrect.

4. **Epoch loop** (`num_epochs` iterations):

- (a) **Batch loop.** Iterate over `make_batches(X_tr, y_tr, batch_size)`. On each batch, run the canonical five-line training step:

`forward` $\rightarrow$ `loss` $\rightarrow$ `zero_grad` $\rightarrow$ `backward` $\rightarrow$ `step`.

Collect each scalar loss value (call `.item()` to detach from the computation graph) in a temporary list.

- (b) **Record epoch loss.** Compute the mean of that list and append it to `train_losses`.
- (c) **Validation accuracy.** Inside a `with torch.no_grad():` block, run the full validation tensor through the model in a single forward pass, call `compute_accuracy`, and append the result to `val_accuracies`. Using `no_grad` here prevents PyTorch from building a computation graph for validation, saving memory and time.

5. **Return** (`model`, `train_losses`, `val_accuracies`).

Verification. `len(train_losses) == len(val_accuracies) == num_epochs`; `train_losses[-1] < train_losses[0]`; final validation accuracy $> 82\%$; all values in `val_accuracies` lie in $[0, 1]$.

$\rightarrow$ When all five tests print $\checkmark$, run `check-cs240` and call a TA.